Disallow Binding a Returned glvalue to a Temporary

Document #:	P2748R0
Date:	2023-01-12
Project:	Programming Language C++
Audience:	Evolution

Reply-to:	Brian Bi <bbi10@bloomberg.net>

Contents

§ 1 Introduction

The following code contains a bug: It initializes a reference from an object of a different type (the programmer has forgotten that the first element of the pair is `const`), resulting in the creation of a temporary. As a result, the reference `d_first` is always dangling:

```
struct X {
    const std::map<std::string, int> d_map;
    const std::pair<std::string, int>& d_first;

    X(const std::map<std::string, int>& map)
        : d_map(map), d_first(*d_map.begin()) {}
};
```

Luckily, the above code is actually ill formed (11.9.3 [class.base.init]/8). But here is a piece of code that contains essentially the same bug:

```
struct Y {
    std::map<std::string, int> d_map;

    const std::pair<std::string, int>& first() const {
        return *d_map.begin();
```

```
    }
};
```

This code is valid, although compilers might warn. Like the first piece of code in this paper, this code snippet always produces a dangling reference. We should make it likewise ill formed.

§ 2 Background

In [CWG1696], Richard Smith pointed out that, while binding a reference member to a temporary in a *mem-initializer* was explicitly called out in the Standard as one of the cases where the lifetime of the temporary is *not* extended to the lifetime of the reference, no corresponding wording was offered for the case in which the expression that produces the temporary is supplied by a default member initializer.

Initially, the proposed resolution simply resolved the inconsistency in favor of explicitly specifying that *brace-or-equal-initializers* behave the same way as *mem-initializers* (i.e., neither extends lifetime). However, at the Issaquah meeting in 2014, making both ill formed was suggested. CWG appears to have accepted this suggestion without controversy. (At the Urbana-Champaign meeting later that year, Issue 1696 was given DR status).

This change was so uncontroversial because binding a reference to a temporary, when the reference will outlive the temporary and become dangling as soon as the full-expression completes, is always a bug. In some simple cases, a novice programmer might not understand that a temporary must be materialized when binding a reference to a prvalue. On the other hand, the examples given in the introduction represent code that experienced C++ developers can easily write.

§ 3 Proposal

Just as the dangling reference created by `X`'s constructor is always a bug, the same is true for the dangling reference created by `Y::first`. In fact, I can imagine some obscure situations in which binding a reference member to a temporary in a *mem-initializer* could be useful to cache the result of an expensive computation, which could then be used by later *mem-initializers* and within the *compound-statement* of the constructor. In contrast, when binding a returned glvalue to a temporary, even such obscure, limited applications seem nonexistent.

I propose, therefore, to make binding a returned glvalue to a temporary ill formed, and I submit that the case for making this change is as strong as — if not stronger than — the case for making binding a reference member to a temporary in a *mem-initializer* ill formed, a decision that apparently did not engender any recorded controversy.

§ 4 Wording

The proposed wording is relative to [N4917].

Strike bullet (6.11) in section 6.7.7 [class.temporary]:

> — ~~The lifetime of a temporary bound to the returned value in a function `return` statement (8.7.4) is not extended; the temporary is destroyed at the end of the full-expression in the `return` statement.~~

Insert a new paragraph, 6, at the end of section 8.7.4 [stmt.return]:

> In a function whose return type is a reference, a `return` statement that binds the returned reference to a temporary expression (6.7.7 [class.temporary]) is ill-formed.
> [*Example 2:*
>
> ```
> auto&& f1() {
>     return 42;  // ill-formed
> }
> const double& f2() {
>     static int x = 42;
>     return x;   // ill-formed
> }
> auto&& id(auto&& r) {
>     return static_cast<decltype(r)&&>(r);
> }
> auto&& f3() {
>     return id(42);  // OK, but probably a bug
> }
> ```
>
> — *end example*]

(*Note:* See [CWG GitHub issue 200] regarding a possible issue with the above wording.)

§ 5 References

[CWG GitHub issue 200] Brian Bi. 2022-12-16. Missing definition of “temporary expression.”
https://github.com/cplusplus/CWG/issues/200

[CWG1696] Richard Smith. 2013-05-31. Temporary lifetime and non-static data member initializers.
http://wg21.link/CWG1696

[N4917] Thomas Köppe. 2022-09-05. Working Draft, Standard for Programming Language C++.
http://wg21.link/N4917